

Nama: Carlos Fadelillah Rama

Kelas: TI-1H

Matkul: Praktikum ASD

2.1.3 Pertanyaan (Percobaan 1)

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

- **Jawaban:** Perbaikan dilakukan pada method `print()` di class `StackTugasMahasiswa`. Agar urutan tampilan sesuai dengan tumpukan (dari yang terbaru di atas ke yang terlama di bawah), perulangan harus berjalan terbalik dari indeks `top` ke 0. Selain itu, tambahkan karakter `\t` yang konsisten agar tampilan kolom Nama, NIM, dan Kelas rapi sesuai hasil verifikasi.

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!

- **Jawaban:** Berdasarkan class `MahasiswaDemo`, jumlah data yang dapat ditampung adalah **5 data**.
- **Potongan kode:**

```
``java StackTugasMahasiswa stack = new StackTugasMahasiswa(5);
```
-

3. Mengapa perlu pengecekan kondisi `isFull()` pada method `push`? Kalau kondisi if-else tersebut dihapus, apa dampaknya?

- **Jawaban:** Pengecekan `isFull()` diperlukan untuk memastikan bahwa array `stack` masih memiliki ruang sebelum data baru ditambahkan. Jika kondisi ini dihapus, program akan tetap mencoba menambah indeks `top` dan memasukkan data ke indeks yang berada di luar kapasitas array (misalnya indeks 5 pada array berukuran 5). Hal ini akan menyebabkan error **`ArrayIndexOutOfBoundsException`** dan program akan berhenti paksa (*crash*).

4. Modifikasi kode program (Lihat Tugas Terbawah):

- **Jawaban: * Pada `StackTugasMahasiswa`:** Tambahkan method `peekBottom()` yang mengembalikan `stack[0]` (indeks pertama/paling bawah).
 - **Pada `MahasiswaDemo`:** Tambahkan pilihan menu baru dan panggil `stack.peekBottom()`.

5. Tambahkan method untuk menghitung banyak tugas:

- **Jawaban: * Method:** Cukup mengembalikan nilai `top + 1`. Karena `top` dimulai dari -1, maka jumlah elemen saat ini adalah indeks `top` ditambah satu.

2.2.3 Pertanyaan (Percobaan 2)

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!

- **Jawaban:** * Method menerima parameter nilai desimal.
 - Selama nilai > 0 , dilakukan operasi modulus ($\% 2$) untuk mendapatkan sisa bagi.
 - Sisa bagi tersebut di-push ke dalam StackKonversi.
 - nilai kemudian dibagi 2 ($/ 2$) untuk iterasi berikutnya.
 - Setelah nilai desimal habis, data diambil dari stack menggunakan pop() satu per satu dan disusun ke dalam String biner. Karena sifat stack LIFO, sisa bagi yang terakhir didapat (bit paling depan) akan keluar lebih dulu.

2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (nilai $\neq 0$), bagaimana hasilnya? Jelaskan alasannya!

- **Jawaban:** Hasilnya akan **tetap sama** untuk bilangan positif. Alasannya, pada bilangan bulat positif, kondisi nilai > 0 dan nilai $\neq 0$ akan terpenuhi selama nilai tersebut belum mencapai 0 setelah dibagi terus-menerus. Namun, perulangan $\neq 0$ secara teknis lebih aman jika sistem harus menangani input bilangan negatif (walaupun dalam konteks nilai tugas, nilai biasanya positif).